



GUÍA DE BUENAS PRÁCTICAS EN GITHUB

Programa de Ingeniería de Sistemas

1. Introducción

GitHub es una plataforma fundamental en el desarrollo de software moderno. No solo permite almacenar código, sino que facilita el trabajo colaborativo, el control de versiones, la revisión de código, la automatización de pruebas y el despliegue de aplicaciones.

Esta guía tiene como objetivo enseñar **buenas prácticas reales**, usadas en la industria, para que los estudiantes desarrollen proyectos con estándares profesionales desde su etapa académica.

2. Organización del repositorio

Un repositorio bien organizado facilita la comprensión del proyecto y el trabajo en equipo.

Estructura recomendada

```
/
├── .github/
│   ├── workflows/           # Automatización (CI/CD)
│   └── PULL_REQUEST_TEMPLATE.md
├── docs/                    # Documentación del proyecto
├── src/                     # Código fuente
├── tests/                   # Pruebas
├── .gitignore
├── README.md
└── LICENSE
```

Buenas prácticas

- Un repositorio debe tener **un propósito claro**.
- No subir archivos innecesarios o temporales.
- Mantener una estructura clara y consistente.
- Separar código, documentación y pruebas.

3. Archivo README.md

El archivo `README.md` es el documento principal del proyecto. Debe permitir que cualquier persona entienda el proyecto sin necesidad de explicaciones adicionales.

Contenido mínimo recomendado

Nombre del proyecto

Descripción

Explicación breve del problema que soluciona el proyecto.

Objetivo

Qué se busca lograr con el sistema.

Tecnologías utilizadas

- Lenguaje de programación
- Frameworks
- Base de datos
- Herramientas adicionales

Instalación y ejecución

Pasos para ejecutar el proyecto localmente.

Uso

Cómo utilizar la aplicación.

Pruebas

Cómo ejecutar las pruebas.

```
## Autores
Nombre de los integrantes del equipo.
```

Buenas prácticas

- Lenguaje claro y sencillo.
- Evitar descripciones ambiguas.
- Mantener el README actualizado.

4. Control de versiones y ramas (Branching)

Git permite manejar versiones del código mediante ramas (branches).

Estrategia recomendada

```
main          → versión estable
develop       → integración de cambios
feature/*     → nuevas funcionalidades
fix/*         → corrección de errores
```

Reglas básicas

- No trabajar directamente sobre la rama `main`.
- Cada funcionalidad debe desarrollarse en una rama propia.
- Usar Pull Requests para integrar cambios.
- Mantener ramas pequeñas y con un solo objetivo.

5. Buenas prácticas en commits

Un commit representa un cambio específico en el proyecto.

Convención recomendada

```
feat: agregar registro de usuarios
fix: corregir error en validación de login
docs: actualizar documentación
refactor: mejorar estructura del código
test: agregar pruebas unitarias
```

Buenas prácticas

- Commits pequeños y frecuentes.
- Mensajes claros y descriptivos.

- Un commit debe resolver **una sola cosa**.
- Evitar mensajes genéricos como “cambios”, “arreglos”, “final”.

6. Pull Requests (PR)

Un Pull Request permite revisar y discutir cambios antes de integrarlos al proyecto principal.

Qué debe incluir un buen Pull Request

- Descripción clara del cambio.
- Motivo del cambio.
- Pasos para probar la funcionalidad.
- Evidencia si aplica (capturas, logs).

Ejemplo de descripción

Descripción:

Se agregó validación de campos obligatorios en el formulario de registro.

Motivo:

Evitar el envío de información incompleta.

Cómo probar:

1. Ingresar al formulario
2. Dejar campos vacíos
3. Verificar mensaje de error

Seguridad en GitHub

La seguridad es responsabilidad de todos los desarrolladores.

Nunca subir al repositorio

- Contraseñas
- Tokens
- Llaves privadas
- Archivos `.env`

Buenas prácticas

- Usar archivos `.gitignore`.
- Manejar variables sensibles fuera del código.
- Revisar el contenido antes de hacer commit.

7. Documentación del proyecto

Un proyecto sin documentación es difícil de mantener.

Qué documentar

- Arquitectura general
- Decisiones técnicas
- Configuración del entorno
- Limitaciones conocidas

Buenas prácticas

- Documentar mientras se desarrolla.
- Usar diagramas sencillos.
- Mantener la documentación actualizada.

8. Trabajo en equipo y ética profesional

GitHub es una herramienta técnica, pero el trabajo en equipo es humano.

Buenas prácticas

- Respetar opiniones técnicas.
- Aceptar retroalimentación.
- Revisar código con criterio constructivo.
- Cumplir estándares acordados.